

O parâmetro **response_model** da **FastAPI** é utilizado para definir qual modelo de dados a função de rota deve retornar. Ele é muito útil para garantir que a resposta da rota seja validada e formatada corretamente de acordo com um esquema específico.

O que é o **response_model**?

- **Validação de Dados:** O **response_model** é utilizado pela **FastAPI** para garantir que a resposta que você está retornando está no formato esperado. Quando a resposta é convertida, a **FastAPI** valida os dados antes de enviá-los ao cliente, assegurando que os dados atendam ao modelo especificado.
- **Documentação Automática:** A **FastAPI** usa o **response_model** para gerar automaticamente a documentação da **API**, como a interface do **Swagger**. Ela mostrará exatamente como a resposta será estruturada.

Tipos Comuns de Modelos no **response_model**:

1. **Pydantic Models:** O modelo mais comum é o modelo **Pydantic**, que é uma biblioteca para validação de dados e criação de esquemas de dados. O **Pydantic** permite definir a estrutura dos dados de maneira clara e concisa.
2. **Listas/Dicionários:** Você pode usar listas ou dicionários com tipos específicos, como listas de objetos ou dicionários onde as chaves e valores são tipados.

Exemplos de Aplicação:

1. Resposta Simples com um Modelo Pydantic

Aqui está um exemplo simples onde a resposta de uma rota **GET** é um objeto de um modelo **Pydantic**, que descreve um curso.

PYTHON

```
from fastapi import FastAPI
from pydantic import BaseModel
```

```
app = FastAPI()
```

```
# Definindo o modelo Pydantic para um Curso
class Curso(BaseModel):
    id: int
    nome: str
    descricao: str
```

```
# Rota GET que retorna um único curso
@app.get("/curso/{curso_id}", response_model=Curso)
async def get_curso(curso_id: int):
    # Aqui estamos apenas criando um exemplo de curso, normalmente seria recuperado do
    # banco de dados.
    curso = {"id": curso_id, "nome": "Curso de Python", "descricao": "Curso básico de
    Python"}
    return curso
```

Explicação:

- **Curso** é o modelo **Pydantic** que define os campos **id**, **nome** e **descricao**.
- A rota **"/curso/{curso_id}"** retorna um objeto do tipo **Curso**.
- A **FastAPI** vai garantir que o retorno tenha o formato esperado: um dicionário com as chaves e tipos definidos em **Curso**.

2. Resposta com Lista de Objetos

Quando você quer retornar uma lista de objetos, pode usar uma lista do modelo **Pydantic** como tipo de resposta.

PYTHON

```
from typing import List
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

# Definindo o modelo Pydantic para um Curso
class Curso(BaseModel):
    id: int
    nome: str
    descricao: str

# Rota GET que retorna uma lista de cursos
@app.get("/cursos", response_model=List[Curso])
async def get_cursos():
    # Lista de cursos
    cursos = [
        {"id": 1, "nome": "Curso de Python", "descricao": "Curso básico de Python"},
        {"id": 2, "nome": "Curso de Django", "descricao": "Curso de desenvolvimento web"},
    ]
    return cursos
```

Explicação:

- **List[Curso]** indica que a resposta será uma lista de objetos do tipo `Curso`.
- Quando a rota `/cursos` for chamada, a resposta será uma lista de cursos, cada um no formato definido pelo modelo `Curso`.

3. Resposta com Dicionário de Objetos

Você também pode retornar um dicionário onde as chaves têm um tipo específico, como inteiros, e os valores são objetos **Pydantic**.

PYTHON

```
from typing import Dict
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

# Definindo o modelo Pydantic para um Curso
class Curso(BaseModel):
    id: int
    nome: str
    descricao: str

# Rota GET que retorna um dicionário de cursos, com ID como chave
@app.get("/cursos_dict", response_model=Dict[int, Curso])
async def get_cursos():
    # Dicionário de cursos, onde a chave é o id do curso
    cursos = {
        1: {"id": 1, "nome": "Curso de Python", "descricao": "Curso básico de Python"},
        2: {"id": 2, "nome": "Curso de Django", "descricao": "Curso de desenvolvimento web"},
    }
    return cursos
```

Explicação:

- **Dict[int, Curso]** indica que a resposta será um dicionário onde as chaves são inteiros (IDs dos cursos) e os valores são objetos do tipo `Curso`.
- Ao acessar a rota `/cursos_dict`, o cliente receberá um dicionário com os cursos organizados pelo ID.

4. Resposta com Dados Padrão (Default)

A FastAPI também permite que você defina um valor padrão ou valores de exemplo diretamente no modelo Pydantic.

PYTHON

```
from pydantic import BaseModel

class Curso(BaseModel):
    id: int = 1
    nome: str = "Curso Default"
    descricao: str = "Descrição padrão do curso"
```

Neste caso, se a função não fornecer um valor para algum campo, ele irá usar o valor padrão definido.

Quando usar `response_model`?

- **Garantir a integridade dos dados:** Ao definir o `response_model`, você garante que os dados retornados pela **API** estarão no formato correto.
- **Gerar documentação automática:** A **FastAPI** usa o `response_model` para gerar a documentação de sua API de forma clara e precisa.
- **Facilidade na manutenção:** Quando você usa modelos **Pydantic**, a estrutura da resposta fica mais clara e você facilita a manutenção do código no longo prazo.